

UNIVERSIDAD BLAS PASCAL

Programación Orientada a Objetos

4Eng

Plataforma de Oportunidades Laborales para Ingenieros

Integrantes	Drako Lusicic Nahuel Pineda Augusto Landerreche
Profesor	César Alejandro Osimani
Año	2026

1. Introducción

4Eng es una plataforma integral orientada a conectar a ingenieros y programadores con oportunidades laborales, incorporando herramientas de inteligencia artificial para optimizar tanto el proceso de búsqueda de empleo como el de selección de candidatos. El sistema fue concebido como proyecto integrador de la materia Programación Orientada a Objetos, aplicando los principios fundamentales de la disciplina en un contexto de aplicación real.

1.1 Motivación

El mercado laboral tecnológico presenta una brecha significativa entre la oferta de profesionales y la capacidad de las empresas para evaluar candidatos de forma eficiente. 4Eng propone reducir esta fricción automatizando el análisis de currículums, generando rankings de candidatos y centralizando la comunicación entre postulantes y empleadores en una única plataforma.

1.2 Objetivos del Proyecto

- Desarrollar una bolsa de trabajo especializada para ingenieros y programadores.
- Integrar análisis de CVs mediante inteligencia artificial (OpenAI GPT-4.5 mini).
- Proveer un portal empresarial con ranking automático de candidatos.
- Implementar un asistente de IA conversacional integrado en la aplicación.
- Aplicar los principios de la Programación Orientada a Objetos en toda la arquitectura del sistema.

2. Arquitectura del Sistema

El sistema sigue una arquitectura cliente-servidor de dos capas principales: una aplicación de escritorio desarrollada en Qt/C++ que actúa como cliente, y un backend RESTful construido con FastAPI/Python que expone la lógica de negocio. La persistencia se gestiona con MySQL/MariaDB en el servidor y SQLite de forma local en el cliente.

2.1 Estructura General

```
4Eng/  
├─ Login/                               # Aplicación Qt Desktop (C++)  
│   ├── mainwindow.cpp/h               # Ventana de login principal  
│   ├── userwindow.cpp/h               # Interfaz para postulantes  
│   ├── empresawindow.cpp/h            # Interfaz para empresas  
│   ├── adminwindow.cpp/h              # Interfaz administrativa  
│   ├── registerwindow.cpp/h           # Ventana de registro  
│   ├── api_client.cpp/h               # Cliente HTTP/API  
│   └─ localdbmanager.cpp/h            # Gestor de BD local SQLite
```

```
└─ vps/backend/                # Backend Python/FastAPI
   └─ app/main.py               # API REST principal
   └─ ia/cv_analyzer.py         # Análisis de CVs con GPT-4.5 mini
   └─ ia/ranking_engine.py      # Motor de ranking de candidatos
   └─ ia/chat_ai.py             # Asistente de chat
```

2.2 Diagrama de Flujo de Datos

El siguiente diagrama resume el flujo de información entre los componentes del sistema:



2.3 Módulos del Backend

Módulo	Responsabilidad
auth/	Registro, login y validación de JWT
profiles/	Gestión de perfiles de ingenieros y empresas
jobs/	CRUD de ofertas laborales
applications/	Gestión del ciclo de vida de postulaciones
cv/	Carga, extracción OCR y análisis de CVs
chat/	Asistente conversacional con IA
ia/ranking_engine	Generación automática de rankings de candidatos

3. Tecnologías Utilizadas

Capa	Tecnología	Función
Frontend	Qt 6 / C++17	Interfaz gráfica multiplataforma
	SQLite	Caché local y sesiones
	QNetworkManager	Cliente HTTP para comunicación con API
Backend	Python 3.11 / FastAPI	API REST principal
	MySQL / MariaDB	Base de datos principal
	Docker	Containerización y despliegue
IA	OpenAI GPT-4.5 mini	Análisis de CVs, ranking y chat

	Tesseract OCR	Extracción de texto de documentos
	PyPDF2 / python-docx	Procesamiento de PDF y DOCX

4. Aplicación de Principios de POO

El diseño del sistema refleja de manera explícita los cuatro pilares de la Programación Orientada a Objetos. A continuación se describe cómo se materializa cada principio en el código del proyecto.

4.1 Encapsulamiento

Cada ventana de la aplicación de escritorio es una clase independiente derivada de `QMainWindow`, que agrupa su propio estado interno (datos de sesión, modelo de datos) y expone únicamente los métodos necesarios para la interacción. Los servicios transversales, como `ApiClient` y `LocalDbManager`, también encapsulan su lógica interna y se comunican con el resto del sistema a través de interfaces bien definidas.

- `ApiClient`: encapsula todas las llamadas HTTP, manejo de tokens JWT y serialización/deserialización JSON.
- `LocalDbManager`: abstrae el acceso a SQLite, exponiendo métodos de alto nivel como `saveSession()` o `getCachedJobs()`.

4.2 Herencia

Las ventanas principales heredan de `QMainWindow` y `QWidget` (clases base de Qt), reutilizando la infraestructura de eventos, renderizado y ciclo de vida que provee el framework. En el backend Python, la herencia de las clases base de FastAPI (`APIRouter`, `BaseModel` de Pydantic) permite estructurar los módulos de forma consistente.

4.3 Polimorfismo

El sistema maneja tres tipos de usuarios distintos (Postulante, Empresa, Administrador), cada uno con su propia ventana e interfaz adaptada. El proceso de autenticación retorna un objeto de usuario que puede tomar cualquiera de estos roles, y el código de la aplicación reacciona polimórficamente para redirigir al usuario a la ventana correspondiente.

En el backend, los distintos motores de análisis de IA (`cv_analyzer`, `ranking_engine`, `chat_ai`) comparten una interfaz común, permitiendo que el router de la API los invoque de forma uniforme independientemente de la implementación subyacente.

4.4 Abstracción

La capa de API actúa como una abstracción entre la lógica de presentación y la lógica de negocio. El cliente Qt no conoce cómo se almacenan los datos ni cómo se procesa un CV: solo conoce los endpoints disponibles y el contrato de datos (JSON). Del mismo

modo, el módulo `ia/` abstrae la comunicación con OpenAI, exponiendo funciones de alto nivel como `analizarCV()` o `generarRanking()` que ocultan los detalles de las llamadas a la API externa.

5. Funcionalidades Principales

5.1 Para Postulantes

- Gestión de perfil personal y profesional.
- Carga de CV en formato PDF o DOCX con procesamiento OCR automático.
- Análisis de CV con IA: score general, resumen, fortalezas, debilidades y skills detectadas.
- Búsqueda y filtrado de ofertas laborales disponibles.
- Postulación a puestos con seguimiento de estado e historial.
- Asistente IA conversacional para asesoramiento profesional.

5.2 Para Empresas

- Gestión de perfil corporativo.
- Publicación y administración de ofertas laborales con descripción de requisitos y nivel de seniority.
- Visualización de candidatos por oferta con acceso a análisis de CV resumido.
- Ranking automático de candidatos con puntuación, análisis de compatibilidad y recomendaciones.
- Acciones sobre postulaciones: aceptar, rechazar, solicitar reunión o enviar mensaje.

5.3 Administración

- Panel de administración para gestión global de usuarios, empresas y ofertas.
- Monitoreo del estado de la plataforma.

6. Endpoints Principales de la API

Método	Endpoint	Descripción
POST	/auth/register	Registro de nuevos usuarios
POST	/auth/login	Autenticación y emisión de JWT
GET	/auth/me	Datos del usuario autenticado
POST	/cv/upload	Subir CV (PDF/DOCX)
POST	/cv/{id}/analyze	Análisis de CV con IA
GET	/jobs	Listar ofertas disponibles

POST	/jobs	Crear nueva oferta laboral
POST	/applications	Crear postulación
PUT	/applications/{id}	Actualizar estado de postulación
POST	/ai/rank-candidates	Generar ranking de candidatos con IA
POST	/ai/chat	Chat con asistente IA

7. Seguridad y Autenticación

El sistema implementa un esquema de seguridad basado en JWT (JSON Web Tokens) con control de acceso por roles. Cada petición al backend es validada contra el token presente en el header Authorization, y los endpoints verifican que el rol del usuario sea el correcto para la operación solicitada.

Mecanismo	Descripción
JWT (JSON Web Tokens)	Tokens de sesión con expiración configurable
Control de acceso por roles	Roles: Postulante, Empresa, Administrador
Hashing de contraseñas	Almacenamiento seguro con algoritmo de hash
SQLite local seguro	Caché de sesión para auto-login sin reenviar credenciales

8. Instalación y Despliegue

8.1 Requisitos Previos

- Qt 6.x instalado en el sistema de desarrollo.
- Python 3.11 o superior.
- MySQL / MariaDB (o Docker para ejecución containerizada).
- Docker y Docker Compose (opcional, recomendado para producción).

8.2 Ejecución del Backend

```
cd vps/backend
pip install -r requirements.txt
uvicorn app.main:app --host 0.0.0.0 --port 8000
```

8.3 Compilación del Cliente Desktop

```
cd Login
qmake 4eng-Login.pro
make
./4eng-Login
```

8.4 Despliegue con Docker

```
cd vps/backend
docker build -t 4eng-backend .
docker run -p 8000:8000 -e DATABASE_URL=<cadena> 4eng-backend
```

8.5 Inicialización de la Base de Datos

Ejecutar el script `vps/db/init.sql` en el motor de base de datos seleccionado antes de levantar el backend. Este script crea todas las tablas y carga los datos iniciales necesarios para el funcionamiento de la plataforma.

9. Estrategia de Ramas (Git Flow)

Rama	Propósito
main	Código estable y funcional listo para entrega
develop	Rama de integración con pruebas en progreso
feature/*	Nuevas características en desarrollo aislado
hotfix/*	Correcciones urgentes sobre main

10. Conclusiones

El desarrollo de 4Eng permitió aplicar de forma práctica y en un contexto real los principios fundamentales de la Programación Orientada a Objetos. La separación en clases y módulos con responsabilidades bien definidas facilitó el trabajo colaborativo del equipo y la mantenibilidad del código a lo largo del proyecto.

La integración de inteligencia artificial mediante la API de OpenAI demostró cómo una arquitectura bien diseñada, con capas de abstracción adecuadas, permite incorporar servicios externos de forma limpia sin acoplar la lógica de negocio a una tecnología específica.

La combinación de Qt/C++ para el frontend y FastAPI/Python para el backend representa un stack tecnológico maduro que cumple con los requisitos de rendimiento, escalabilidad y facilidad de desarrollo para el tipo de aplicación propuesta.

Los objetivos académicos planteados al inicio del proyecto fueron alcanzados satisfactoriamente, resultando en una plataforma funcional que integra gestión de usuarios, publicación de ofertas laborales, análisis automático de currículums y asistencia por inteligencia artificial.